ENSO Finance ~ Flashloan Adapters

Smart Contract Security Assessment

February 02, 2026

ABSTRACT

Dedaub was commissioned to perform a security audit of the flashloan adapters to be added within the Enso Shortcuts ecosystem. The intent behind the newly introduced functionality is for users to be able to compose complex on-chain actions powered by the Enso Shortcut system within the context of flashloan operations.

SETTING & CAVEATS

This audit report mainly covers the contracts located in the public repository of the protocol https://github.com/EnsoBuild/shortcuts-client-contracts, on branch `flashloan` at commit `4aa5579d3c1f1d1c6f9792abad2c85386f55c0ee`. The changes can be collectively viewed at the following pull request: https://github.com/EnsoBuild/shortcuts-client-contracts/pull/26

Audit Start Date:	January 29, 2026
Audit End Date:	January 30, 2026
Report Submission Date:	February 02, 2026

2 auditors worked on the following contracts:

```
src/
├── wallet/
│   └── EnsoWalletV2.sol
│
└── flashloan/
    ├── AbstractEnsoFlashloan.sol
    ├── EnsoRouterFlashloanAdapter.sol
```

```
├── EnsoSafeFlashloanAdapter.sol
└── EnsoWalletFlashloanAdapter.sol
```

The audit's main target is security threats, i.e., what the community understanding would likely call "hacking", rather than the regular use of the protocol. Functional correctness (i.e. issues in "regular use") is a secondary consideration. Typically it can only be covered if we are provided with unambiguous (i.e. full-detail) specifications of what is the expected, correct behavior. In terms of functional correctness, we often trusted the code's calculations and interactions, in the absence of any other specification. Functional correctness relative to low-level calculations (including units, scaling and quantities returned from external protocols) is generally most effectively done through thorough testing rather than human auditing.

The introduced functionality directly integrates with various protocols that offer flashloan capabilities:

- Morpho
- Aave V3
- Balancer V3
- Dolomite

This meant that the auditors expended significant effort towards understanding all the security assumptions of each third party protocol and determining whether the security of the protocol can be impacted.

VULNERABILITIES & FUNCTIONAL ISSUES

This section details issues affecting the functionality of the contract. Dedaub generally categorizes issues according to the following severities, but may also take other considerations into account such as impact or difficulty in exploitation:

Category	Description

CRITICAL	Can be profitably exploited by any knowledgeable third-party attacker to drain a portion of the system's or users' funds OR the contract does not function as intended and severe loss of funds may result.
HIGH	Third-party attackers or faulty functionality may block the system or cause the system or users to lose funds. Important system invariants can be violated.
MEDIUM	Examples: • User or system funds can be lost when third-party systems misbehave. • DoS, under specific conditions. • Part of the functionality becomes unusable due to a programming error.
LOW	Examples: • Breaking important system invariants but without apparent consequences. • Buggy functionality for trusted users where a workaround exists. • Security issues which may manifest when the system evolves.

Issue resolution includes "dismissed" or "acknowledged" but no action taken, by the client, or "resolved", per the auditors.

CRITICAL SEVERITY:

[No critical severity issues]

HIGH SEVERITY:

[No high severity issues]

MEDIUM SEVERITY:

[No medium severity issues]

LOW SEVERITY:

[No low severity issues]

OTHER / ADVISORY ISSUES:

This section details issues that are not thought to directly affect the functionality of the project, but we recommend considering them.

ID	Description	STATUS
A1	Important security check downplayed in comments	**INFO**

From the following comment:

`AbstractEnsoFlashloan.sol::uniswapV3FlashCallback:402`

```
...
IUniswapV3Pool pool = IUniswapV3Pool(msg.sender);
address factory = pool.factory();
_verifyLender(factory, LenderProtocol.UniswapV3);

// NOTE: this might be redundant since we get the factory to verify lender
uint24 poolFee = pool.fee();

address expectedPool = IUniswapV3Factory(factory).getPool(params.token0,
params.token1, poolFee);
if (msg.sender != expectedPool) {
    revert UnknownLender();
}
...
```

It might be implied that the check verifying that a caller of the `AbstractEnsoFlashloan::uniswapV3FlashCallback` function is a valid UniswapV3 pool deployed by the trusted factory is not needed.

However, this is not true. The above-mentioned check ensures that a caller cannot bypass the callback authorization by merely implementing a `factory()` method that returns an approved factory address.

A2	Tokens with on-transfer fees are implicitly not supported	**INFO**

Stemming from the following `EnsoRouterFlashloanAdapter` snippets:

`EnsoRouterFlashloanAdapter.sol::executeShortcut:38`

```
...
balanceBefore = IERC20(token).balanceOf(address(this)) - amount;
...
```

`EnsoRouterFlashloanAdapter.sol::executeShortcutMulti:67`

```
...
balancesBefore[i] = IERC20(tokens[i]).balanceOf(address(this)) - amounts[i];
...
```

It's not possible to integrate with tokens that feature on-transfer fees. When such tokens are used within the context of a flashloan, the amount that is actually transferred is practically less than the requested amount in the `amount` and `amount[i]` values – therefore causing the transaction to revert.

This is not a direct issue for the `EnsoWalletFlashloanAdapter` and `EnsoSafeFlashloanAdapter` contracts so this informatory note is raised in the event the protocol cares about integrating with tokens featuring on-transfer fees.

A3	Integration-level security assumptions can potentially be further strengthened	**INFO**

A major point of concern for the integration with various flashloan providers is that if a vulnerability in the contracts of the third party protocol is ever discovered in the future , it might allow an an attacker to invoke the flashloan callbacks directly from the vulnerable third party contracts:

AbstractEnsoFlashloan.sol::onMorphoFlashLoan:282

```
function onMorphoFlashLoan(uint256 amount, bytes calldata data) external {
    _verifyLender(msg.sender, LenderProtocol.Morpho);

    (
        address wallet,
        IERC20 token,
        bytes32 accountId,
        bytes32 requestId,
        bytes32[] memory commands,
        bytes[] memory state
    ) = abi.decode(data, (address, IERC20, bytes32, bytes32, bytes32[], bytes[]));

/*
  @Dedaub:

  Assume that a vulnerability on the morpho contract allowed someone to call into
the adapter with arbitrary data
*/
    uint256 balanceBefore = executeShortcut(wallet, accountId, requestId,
commands, state, address(token), amount);
...
```

This is a catastrophic scenario, since this would allow an unauthorized party to invoke arbitrary Shortcut instructions within the context of a user's wallet, ultimately putting the users' funds at risk.

This vector had been already identified by the development team: https://github.com/EnsoBuild/shortcuts-client-contracts/pull/26#discussion_r2687268660 and while the auditors spent time reviewing the third party contracts to rule out such an attack, the protocol's decision to guard against such a scenario is deemed correct.

The current guard against this scenario is a reactive one, with the owner of the adapter contracts being able to remove support for a particular flashloan provider:

`AbstractEnsoFlashloan.sol::removeLender:53`

```
function removeLender(address lender) external onlyOwner {
  trustedLenders[lender] = LenderProtocol.None;
  emit LenderRemoved(lender);
}
```

A more proactive defensive approach against the same scenario would involve:

1. The addition of a guard variable that is set when the adapter contract initiates a flashloan via `AbstractEnsoFlashloan::executeFlashloan`
2. Checking whether the above-mentioned guard has been set when a flashloan callback is invoked.

Under this scheme, even if a future vulnerability enables unauthorized invocation of a flashloan callback, the adapter contracts will reject any calls that do not originate from `AbstractEnsoFlashloan::executeFlashloan`. This ensures that user wallets remain protected from such attack vectors.

With the introduction of transient storage in the latest EVM versions, such a guard will introduce only a really small (near-negligible) gas overhead.

A4	Compiler bugs	**INFO**

While the source code supports all versions of `0.8.20` versions and above within the same minor version, the code is compiled with Solidity `0.8.28`. Version `0.8.28`, in particular, has some known bugs, which we do not believe affect the correctness of the contracts.

DISCLAIMER

The audited contracts have been analyzed using automated techniques and extensive human inspection in accordance with state-of-the-art practices as of the date of this report. The audit makes no statements or warranties on the security of the code. On its own, it cannot be considered a sufficient assessment of the correctness of the contract. While we have conducted an analysis to the best of our ability, it is our recommendation for high-value contracts to commission several independent audits, a public bug bounty program, as well as continuous security auditing and monitoring through Dedaub Security Suite.

ABOUT DEDAUB

Dedaub offers significant security expertise combined with cutting-edge program analysis technology to secure some of the most prominent protocols in DeFi. The founders, as well as many of Dedaub's auditors, have a strong academic research background together with a real-world hacker mentality to secure code. Protocol blockchain developers hire us for our foundational analysis tools and deep expertise in program analysis, reverse engineering, DeFi exploits, cryptography and financial mathematics.